

RAP 後端開發練習 8：Associations / Compositions (多層結構)

Lecture

這一課要解決什麼問題，以及為什麼這一課的建立方式跟前面不一樣

rap02 ~ rap07 全部都是「單一實體」的 RAP BO——一張表對一個 CDS View 對一組 Behavior。但真實業務資料幾乎都是**多層結構**：一張訂單 (Header) 底下有好幾筆訂單項目 (Item)，刪除訂單要連帶刪除所有項目，新增訂單常常要「連同項目一次送出」。RAP 用 **Composition (組合)** 表達這種父子關係，用 **Association (關聯)** 讓子實體反向指回父實體。

這一課開始，物件建立方式改變：rap01 ~ rap07 的主要課程物件都是 Claude 用 ADT API 直接建立 ([.claude/rules/sap-adt-mcp.md](#) 記錄的 workaround)，但這是課程一開始就講好的**教學分工原則** (見 README)——這樣的教法有個缺口：如果自始至終都是 Claude 建物件，你不會練到「用 Eclipse 精靈實際建立一組 RAP 物件」這個真實工作最常用的操作。rap05 ~ rap07 因為都是在既有物件上加欄位、加語法，還可以沿用「Claude 建立、你事後核對」的模式；但這一課要建**全新的兩張表 + 兩個 CDS View + 一個 BDEF + 一個實作類別**，份量跟複雜度都達到「應該讓你自己動手建」的門檻——這一課的主要物件，改成由你在 **Eclipse ADT** 建立，**Claude** 負責事後驗證 (讀取比對、語法檢查) 與除錯。

不過在把步驟交給你之前，Claude 已經用暫時性的驗證物件 (**scratch objects**，跟正式課程物件不同名，事後會留在 **\$TMP** 但不算進課程物件清單) 把這一課要教的語法在這個系統上逐字驗證過一輪——包括過程中踩到的兩個跟官方教材不一樣的地方 (下面會詳細說明)，確保接下來給你的步驟跟程式碼是「這個系統上真的能動」的版本，不是照抄官方文件、你自己去試錯。

⚠ **更正 (2026-08-17)**：這篇講義原本第 1 / 2 步的 Table DDL 把 `order_id / item_id / quantity` 三個欄位寫成內建型別 (`abap.char(10) / abap.char(4) / abap.dec(9,2)`)，違反了本課程一貫要求「DDIC 欄位一律要引用 Data Element、不可留內建型別」的規則 (見 [CLAUDE.md](#) / [.claude/rules/abap-style.md](#))——這是 Claude 寫這篇講義時自己的疏漏，由使用者在 Eclipse 實際建立 `ZRAP08_ORDER` 表格時發現並回報。這三個欄位跟 rap02 的 `task_id` 一樣是「課程獨有的業務概念」(不是真實 SAP 標準單據的欄位，找不到語意相符的標準 Data Element)，所以比照 rap02 的做法，Claude 已建立三組專屬 Domain + Data Element (`ZRAP08_ORDERID / ZRAP08_ITEMID / ZRAP08_QUANTITY`，皆已啟用)，下面第 1 / 2 步的 DDL 已經更新成引用這三個 DE。如果你已經照舊版講義建好 `ZRAP08_ORDER` 表格，只要把 `order_id` 那一行改成 `key order_id : zrap08_orderid not null;` 重新啟用即可 (`ZRAP08_ORDER_I` 表格如果還沒建，直接用下面新版 DDL；如果已經建了，`order_id / item_id / quantity` 三行都要一併更新)。

Part A：Managed Composition 語法 (知識儲備，這系統無法執行，不要求你建立)

在看語法之前先說明一下：這一課的 Part A 跟 rap05 ~ rap07 的 Part A 一樣，純粹是語法知識儲備 (Managed CUD 這系統執行不了，原因見 rap03 Part C)，這裡不要求你在 Eclipse 建立對應物件——你只需要看懂語法、知道跟 Unmanaged 版本的差異即可。

Composition / Association 在 Managed BDEF 的語法元素 (查證官方文件並用 Claude 的驗證物件實測確認)：

- **define root view vs. define view (少了 root)**：這個差異不是隨便選的，它決定這個 CDS View 在整個 RAP BO 裡的**角色**：
 - **root**：這個 View 是 BO 的**根節點**，獨立存在、自己管理鎖定，對應的 BDEF 要寫 **lock master**，並且一定要加 **@AbapCatalog.preserveKey: true + @ObjectModel.compositionRoot: true** 這兩個 annotation (rap01 已經踩過的坑，缺了啟用會直接失敗)。一個 RAP BO 只能有一個 root。
 - **不帶 root**：這個 View 是**依附 / 子節點 (Dependent Entity)**，透過 Composition 被父節點「擁有」，自己不管鎖，對應的 BDEF 要寫 **lock dependent (<子鍵> = <父鍵>)**——借用父節點的鎖，並且要用 **association to parent** 反向指回根節點。
 - 這一課的 **ZI_RAP08_ORDER (Header)** 是根節點、用 **define root view**；**ZI_RAP08_ORDER_I (Item)** 是子節點、用 **define view (不帶 root)**——下面第 3 / 4 步 Eclipse 建立時，兩者選的 Templates 雖然 (因為這系統不支援新式 **entity** 語法) 都是同一個 **Define View (obsolete)** 模板，但貼上去的 DDL 內容一個有 **root** 一個沒有，就是靠這個關鍵字區分角色。
- ****composition [0..*] of <子CDS View> as _<別名>****：寫在 CDS Root View 的 **as select from ...** 之後、**{ }** 之前，宣告這個實體「擁有」哪個子實體、關聯數量上限 (**[0..*]** 代表零到多筆)。這是這系統既有標準物件 **C_SalesOrderManage** 本身就在用的語法 (**composition [0..*] of C_SalesOrderItemManage as _Item**)，不是新式 **view entity** 專屬的語法，這系統的舊式 **define view / define root view** 完全支援。
- **association to parent <父CDS View> as _<別名> on \$projection.<子鍵> = _<別名>.<父鍵>**：寫在子實體 CDS View 裡，宣告「回指父實體」的關聯，**to parent** 是固定關鍵字。
- **BDEF 裡 association _<別名> { create; }**：宣告父實體允許透過這個關聯建立子實體 (Create-by-Association，簡稱 CBA)——這是這一課的核心機制：呼叫端可以「建立 Header 的同時，一次把底下的 Item 也建立好」，不用分兩次呼叫。
- **⚠ 子實體的 lock 子句，這系統要用 lock dependent (<子鍵> = <父鍵>)**，不是官方教材常見的 **lock dependent by _<別名>**——Claude 用驗證物件實測：**lock dependent by _Header** 啟用報 " (" expected, not "by".，改成 **lock dependent (order_id = order_id)** 才編譯成功。子實體不用自己管理鎖定，鎖定跟著父實體走，這個子句只是告訴框架「透過哪個欄位對應找到父實體的鎖」。

驗證過的完整 Managed BDEF (Claude 的驗證物件，語法已確認正確，這系統執行不了)：

```
managed;

define behavior for ZI_RAP08_ORDER alias Header
persistent table zrap08_order
lock master
{
  create;
  update;
  delete;

  association _Item { create; }

  field ( mandatory ) description;
  field ( readonly ) created_at, created_by;
```

```

    mapping for zrap08_order corresponding;
}

define behavior for ZI_RAP08_ORDER_I alias Item
persistent table zrap08_order_i
lock dependent ( order_id = order_id )
{
    update;
    delete;

    field ( readonly ) order_id, item_id;
    field ( mandatory ) material_desc, quantity;

    association _Header;

    mapping for zrap08_order_i corresponding;
}

```

一個 **BDEF** 檔案可以包含多個 **define behavior for** 區塊——這是延伸「多層結構」的自然結果：Header 跟 Item 的行為定義都寫在同一個 BDEF 物件裡（綁在 Root CDS View 上），不是各自獨立的 BDEF 物件。

Part B：Unmanaged Composition + Create-by-Association——這系統真正能跑的版本

跟 rap05 / rap06 不同（Determination / Validation 在非 Draft 的 Unmanaged 完全沒有宣告式語法可用）、也跟 rap07 一樣（Action 沒有這個限制）——**Composition / Association** 的宣告語法，**Managed / Unmanaged** 都一樣（**composition [0..*] of / association to parent / association _Item { create; }**），差別只在**實作邏輯要自己寫**。Claude 已經用驗證物件把 Unmanaged 版本從語法到執行完整跑過一輪，過程中踩到一個這系統特有、官方文件沒有明講的關鍵坑（見下方）。

Unmanaged BDEF 語法（沿用 Part A 的規則，**implementation unmanaged in class ... unique; header**）：

```

implementation unmanaged in class zbp_i_rap08_order unique;

define behavior for ZI_RAP08_ORDER alias Header
lock master
{
    create;
    update;
    delete;

    association _Item { create; }

    field ( readonly ) created_at, created_by;
}

```

```

define behavior for ZI_RAP08_ORDER_I alias Item
lock dependent ( order_id = order_id )
{
    update;
    delete;

    field ( readonly ) order_id;

    association _Header;
}

```

⚠ 這裡有個容易忽略的細節：子實體的 **item_id** (子實體自己的主鍵，使用者建立時要指定) 不能標 **field (readonly)**——只有 **order_id** (從父實體帶過來的外鍵) 才該是 **readonly**。Claude 一開始兩個都標了 **readonly**，啟用時遇到 **The field "ITEM_ID" of entity "... cannot be modified.**——因為 Create-by-Association 建立子實體時，**item_id** 是使用者透過 EML 傳進來的資料，不是框架自動決定的值，標成 **readonly** 會直接擋下這次建立。**判斷原則**：**readonly** 該標在「使用者不該手動指定、由框架或你的程式碼決定值」的欄位 (**order_id** 從父實體帶過來、**created_at/created_by** 由邏輯自動填)，使用者需要親自輸入的欄位 (**item_id**、**material_desc**) 不能是 **readonly**。

Create-by-Association 的 Handler Method：新的 **FOR CREATE <alias>_<association>** 語法

```

METHODS create_item FOR MODIFY
IMPORTING it_create FOR CREATE Header\_Item.

```

- **FOR CREATE Header_Item : _Item** 這個寫法 (反斜線 + 底線開頭的關聯別名) 代表「這是透過 **_Item** 這個關聯觸發的建立」，不是父實體自己的一般 **create**——這是全新的衍生型別，跟 rap03 教過的 **FOR CREATE <alias>** (父實體自己的一般建立) 不一樣。反斜線是 ABAP 語法規則：識別字如果要用地線開頭的名稱 (這裡是關聯別名 **_Item**)，要加反斜線跳脫。

方法本體：

```

METHOD create_item.
    LOOP AT it_create INTO DATA(ls_create).
        LOOP AT ls_create-%target INTO DATA(ls_target).
            INSERT zrap08_order_i FROM @( VALUE #(
                client      = sy-mandt
                order_id     = ls_create-%key-order_id
                item_id      = ls_target-item_id
                material_desc = ls_target-material_desc
                quantity     = ls_target-quantity ) ).
        ENDLOOP.
    ENDLOOP.
ENDMETHOD.

```

- **it_create** 是雙層結構：外層一列代表「對哪一個父實體做 Create-by-Association」，**ls_create-%key-order_id** 是這個父實體的 Key（要用哪個父訂單）；內層 **ls_create-%target** 是一個表格，裝著「這次要為這個父實體建立的所有子實體資料」，每一列對應一筆要新增的 Item。兩層 **LOOP** 是這個 **Handler** 的固定寫法：外層走過所有父實體，內層走過每個父實體要建立的所有子實體。

⚠️⚠️ 這系統踩到的關鍵坑：EML 呼叫端要用 **%key**，不是官方範例常見的 **%cid_ref**

官方文件 ([ABENBDL_DETERMINATION_ABEXA](#)) 的 Create-by-Association 範例，父實體用 **%cid** (暫時代碼) 建立、子實體那段用 **%cid_ref** 反查父實體：

```
" 官方範例寫法 (這系統測試失敗，父實體是框架自動編號才需要這樣)
ENTITY SalesOrder CREATE BY \_SalesOrderItem
  WITH VALUE #( ( %cid_ref = '1' %target = VALUE #( ... ) ) )
```

Claude 一開始照抄這個寫法，語法編譯完全正常、**EML** 呼叫也沒有回報任何錯誤 (**sy-subrc = 0**)，但子實體完全沒有被建立——**FAILED** / **REPORTED** / **MAPPED** 三個回應參數全部是空的，好像什麼事都沒發生。這是這一課排錯過程中最容易誤判的地方：沒有任何錯誤訊息，卻沒有效果。Claude 用一個「哨兵值」技巧（在 Handler Method 裡先無條件插入一筆固定資料，確認方法本身有沒有被呼叫到）才排查出：Handler Method 有被呼叫，但傳進來的 **it_create** 是空表——問題出在 EML 呼叫端「父子關聯」沒有正確建立起來，不是 Handler 邏輯錯。

原因分析：**%cid_ref** 存在的意義是「父實體的正式 Key 要等存檔後才知道」（例如 Managed BDEF 搭配 **numbering:managed** 自動編號），所以子實體建立時只能先用暫時代碼 **%cid_ref** 反查，等真正的 Key 確定後框架才幫你接起來。但這一課的 **order_id** 是使用者自己指定的值，一開始就知道真正的 Key 是什麼，根本不需要透過 **%cid_ref** 反查——改用 **%key-order_id** 直接指定父實體的正式 **Key**，問題就解決了：

```
MODIFY ENTITIES OF zi_rap08_order
  ENTITY Header
    CREATE FIELDS ( description )
    WITH VALUE #( ( %cid = 'H1' order_id = 'ORD001' description = 'Demo Order' ) )

  ENTITY Header
    CREATE BY \_Item
    FIELDS ( item_id material_desc quantity )
    WITH VALUE #( ( %key-order_id = 'ORD001'
      %target = VALUE #(
        ( %cid = 'I1' item_id = '0010' material_desc = 'Widget A'
          quantity = '5' )
        ( %cid = 'I2' item_id = '0020' material_desc = 'Widget B'
          quantity = '3' ) ) ) )

  FAILED DATA(ls_failed)
  REPORTED DATA(ls_reported).

COMMIT ENTITIES.
```

一句話記憶：父實體的 Key 是框架自動編號才需要 %cid_ref；父實體的 Key 是使用者自己指定（這一課的情況），直接用 %key-<欄位> 就好，不用繞 %cid_ref 這一層。這是官方文件沒有明講、只能靠實測才會發現的細節——本質上不是語法錯誤（兩種寫法都編譯得過），是「語意選錯」導致框架靜默地找不到要建立在哪個父實體底下。

✅ 驗證結果 (programrun 無頭執行，完全成功)

Claude 的驗證物件實測輸出：

```
after commit entities
header found: V001      Verify Order
item rows found: 2
0010 Widget A  5.00
0020 Widget B  3.00
```

一次 EML 呼叫 (ENTITY Header CREATE + ENTITY Header CREATE BY _Item)，Header 跟兩筆 Item 同時建立成功——這就是「巢狀 CRUD」的效果：呼叫端完全不用先建 Header、拿到 Key 之後再分開建 Item，一次搞定。

⚠ 為什麼驗證用的是 ABAP Report，不是真正的 OData / 前端呼叫？——RAP 測試的兩層架構

上面這支驗證程式（後面 Step 8 會請 Claude 建立的 ZR_RAP08_DEMO）用的是 EML (MODIFY ENTITIES / READ ENTITIES / COMMIT ENTITIES)——這是 ABAP 語言內建的語法，直接跟 RAP BO 對話，完全不經過 OData / HTTP。這是刻意的設計，不是「懶得測前端」：RAP 開發本來就分成兩個獨立的層次，要分開驗證：

層次	測的是什麼	怎麼測
Behavior / BO 層	Behavior Definition + 實作類別的邏輯本身 (CRUD / Composition / Determination / Validation / Action 對不對)	EML——直接跟 BO 對話，不需要 Service Definition / Binding，可以用 programrun 無頭驗證
Service / OData 層	Service Definition 的 expose、Service Binding 有沒有正確 Publish、OData 協定轉換 (JSON 序列化、CSRF Token、URL 路由) 對不對	真正的 OData 呼叫——Fiori Elements Preview、Postman、瀏覽器

為什麼先測 Behavior 層，這樣做的好處：

- 職責分離，好排錯：如果一開始就直接測 OData，一旦失敗，很難分辨是「BO 邏輯寫錯」還是「Service 曝露設定錯」。先用 EML 把 BO 邏輯鎖定沒問題，之後測 OData 如果失敗，就能直接鎖定問題在 Service 層，不用兩邊一起懷疑。
- 不需要 Service Definition / Binding 就能測：EML 直接對 BO 說話，完全跳過「要先建 SRVD / SRVB / Publish」這一整串前置作業——這也是為什麼 rap08 前面的部分完全沒提到 Service Definition / Binding，先前那個「Create-by-Association 端對端成功」的驗證，測的純粹是 Behavior Definition + 實作類別這一層。
- 這個系統上，OData 層本身有額外的已知障礙，跟 BO 邏輯對不對無關：[.claude/rules/sap-adt-mcp.md](#) 第 45 節記錄過，Claude 端自我呼叫 OData 服務會卡在系統資源競爭或 CSRF Token 驗證失敗

——這些是 OData 層特有的問題。用 EML 讓 Claude 可以透過 `programrun` 無頭、可靠地驗證 BO 邏輯，不會被這些 OData 層的雜訊干擾。

下面先照原本規劃完成 Behavior 層的 6 個物件；**Part D**（在練習之後）會補上 **Service Definition / Service Binding**，讓你能實際測到 **Service / OData** 這一層，用的就是 rap04 已經教過的 Eclipse Publish + Preview 流程。

Eclipse ADT：建立這一課的物件——Step by Step（輪到你了）

這一課換你動手建立主要物件，跟着下面步驟操作，Claude 已經用驗證物件確認語法完全正確，照抄不會遇到上面記錄過的坑：

1. 建立 Header 表格 `ZRAP08_ORDER`

沿用 rap02 教過的「Eclipse ADT 建立 Transparent Table」流程：對著套件 `$TMP` 右鍵 → **New** → **Other ABAP Repository Object** → 搜尋 **Database Table** → Name 填 `ZRAP08_ORDER`、Description 填 `RAP08 Order Header Table` → Finish，貼上以下 DDL：

```
@EndUserText.label : 'RAP08 Order Header Table'
@AbapCatalog.enhancementCategory : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED
define table zrap08_order {
  key client      : mandt not null;
  key order_id    : zrap08_orderid not null;
  description     : text100;
  created_at      : timestamp;
  created_by      : syuname;
}
```

存檔（Ctrl+S）+ 啟用（Ctrl+F3 或工具列 Activate 按鈕）。

2. 建立 Item 表格 `ZRAP08_ORDER_I`

同樣方式，Name 填 `ZRAP08_ORDER_I`：

```
@EndUserText.label : 'RAP08 Order Item Table'
@AbapCatalog.enhancementCategory : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED
define table zrap08_order_i {
  key client      : mandt not null;
  key order_id    : zrap08_orderid not null;
  key item_id     : zrap08_itemid not null;
```

```

material_desc : text100;
quantity      : zrap08_quantity;

}

```

存檔 + 啟用。

3. 建立 Header CDS View **ZI_RAP08_ORDER**

沿用 rap02 教過的「Eclipse ADT 建立 CDS View」流程：對著 **ZRAP08_ORDER** 表格右鍵 → **New Data Definition** (或 **\$TMP** 右鍵 → New → Data Definition) → Name 填 **ZI_RAP08_ORDER**、Description 填 **RAP08 Order Header View** → 精靈的 Templates 畫面選 **Define View** (不要選帶 **entity** 字樣的版本，這系統不支援，rap02 已經教過這個陷阱) → Finish，貼上：

```

@AbapCatalog.sqlViewName: 'ZIRAP08ORDER'
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RAP08 Order Header View'
@ObjectModel.compositionRoot: true
@Metadata.allowExtensions: true
define root view ZI_RAP08_ORDER
  as select from zrap08_order

  composition [0..*] of ZI_RAP08_ORDER_I as _Item
{
  key order_id,
  description,
  created_at,
  created_by,

  _Item
}

```

先不要急著啟用——這裡引用了 **ZI_RAP08_ORDER_I**，這個物件還不存在，啟用會失敗，等下一步建好再一起啟用。

4. 建立 Item CDS View **ZI_RAP08_ORDER_I**

同樣方式，Name 填 **ZI_RAP08_ORDER_I**：

```

@AbapCatalog.sqlViewName: 'ZIRAP08ORDERI'
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'RAP08 Order Item View'
@Metadata.allowExtensions: true
define view ZI_RAP08_ORDER_I

```



```

as select from zrap08_order_i

association to parent ZI_RAP08_ORDER as _Header
on $projection.order_id = _Header.order_id
{
    key order_id,
    key item_id,
    material_desc,
    quantity,

    _Header
}

```

存檔。現在兩個 **CDS View** 互相引用，回到 **ZI_RAP08_ORDER** 一起啟用（Eclipse 通常會自動偵測相依物件，跳出的清單裡勾選兩個一起 Activate；如果沒有自動偵測，兩個都手動各自 Activate 一次，第二次啟用會成功）。

5. 建立 Behavior Definition **ZI_RAP08_ORDER**

沿用 rap03 教過的「Eclipse ADT 建立 Behavior Definition」流程：對著 **ZI_RAP08_ORDER**（Header CDS View）右鍵 → **New Behavior Definition** → Name 固定跟 CDS View 同名（灰的不能改）→ **Implementation Type** 選 **Unmanaged** → Next → Transport（**\$TMP** 直接 Finish）。精靈生成的骨架會有大量註解掉的範例行，整份清空改貼：

```

implementation unmanaged in class zbp_i_rap08_order unique;

define behavior for ZI_RAP08_ORDER alias Header
lock master
{
    create;
    update;
    delete;

    association _Item { create; }

    field ( readonly ) created_at, created_by;
}

define behavior for ZI_RAP08_ORDER_I alias Item
lock dependent ( order_id = order_id )
{
    update;
    delete;

    field ( readonly ) order_id;
}

```

```
association _Header;  
}
```

存檔 + 啟用。

6. 用 **Ctrl+1** 生成實作類別骨架

游標點在 header 那一行的類別名稱 **zbp_i_rap08_order** 上 → 按 **Ctrl+1** → 選 **Create behavior implementation class** 並套用 → 選傳輸請求 (**\$TMP** 免選) → Finish。

⚠ 這份 **BDEF** 有兩個 **define behavior for** 區塊 (**Header + Item**)，系統會依此生成兩個各自獨立的 **Handler** 類別，不是只有一個：**lhc_header** (對應 Header 的 **create / update / delete / association _Item { create; }**) 跟 **lhc_item** (對應 Item 的 **update / delete**)，兩個類別都在同一份 Local Types Include 裡。**lhc_item** 不會有 **FOR LOCK** 方法——延續 Part A 已經講過的規則，**lock dependent** 的子實體不用自己管理鎖定，框架會自動透過 **lock dependent (order_id = order_id)** 這個對應關係去借用 Header 的鎖，所以子實體的 Handler 類別不需要 (也不能) 宣告 **FOR LOCK**。

7. 補上方法本體邏輯

在生成的骨架裡 (Local Types Include)，把每個空方法填上邏輯——**Claude** 已經完整驗證過下面這份程式碼，照抄即可：

```
CLASS lhc_header DEFINITION INHERITING FROM cl_abap_behavior_handler.  
  PRIVATE SECTION.  
    METHODS lock FOR LOCK  
      IMPORTING it_lock FOR LOCK Header.  
  
    METHODS create FOR MODIFY  
      IMPORTING it_create FOR CREATE Header.  
  
    METHODS create_item FOR MODIFY  
      IMPORTING it_create FOR CREATE Header\_Item.  
  
    METHODS read_header FOR READ  
      IMPORTING it_read FOR READ Header RESULT et_result.  
ENDCLASS.  
  
CLASS lhc_header IMPLEMENTATION.  
  
  METHOD lock.  
  ENDMETHOD.  
  
  METHOD create.  
    LOOP AT it_create INTO DATA(ls_create).  
      INSERT zrap08_order FROM @( VALUE #(  
        client      = sy-mandt
```

```

        order_id      = ls_create-order_id
        description    = ls_create-description
        created_at     = cl_abap_tstmp=>utclong2tstmp( utclong_current( ) )
        created_by     = sy-uname ) ).
    ENDLOOP.
ENDMETHOD.

METHOD create_item.
    LOOP AT it_create INTO DATA(ls_create).
        LOOP AT ls_create-%target INTO DATA(ls_target).
            INSERT zrap08_order_i FROM @( VALUE #(
                client      = sy-mandt
                order_id     = ls_create-%key-order_id
                item_id      = ls_target-item_id
                material_desc = ls_target-material_desc
                quantity      = ls_target-quantity ) ).
        ENDLOOP.
    ENDLOOP.
ENDMETHOD.

METHOD read_header.
    LOOP AT it_read INTO DATA(ls_key).
        SELECT SINGLE order_id, description, created_at, created_by
            FROM zrap08_order WHERE order_id = @ls_key-order_id INTO @DATA(ls_data).
        IF sy-subrc = 0.
            APPEND VALUE #(
                %key      = ls_key-%key
                order_id   = ls_data-order_id
                description = ls_data-description
                created_at  = ls_data-created_at
                created_by  = ls_data-created_by ) TO et_result.
        ENDIF.
    ENDLOOP.
ENDMETHOD.

ENDCLASS.

```

⚠ 別漏了 **lhc_item**——這是另一個獨立的類別，對應 **Item** 的 **BDEF** 區塊，一樣寫在同一份 Local Types Include 裡（跟 **lhc_header** 平行，不是巢狀在裡面）：

```

CLASS lhc_item DEFINITION INHERITING FROM cl_abap_behavior_handler.
    PRIVATE SECTION.
        METHODS read_item FOR READ
            IMPORTING it_read FOR READ Item RESULT et_result.

        METHODS update FOR MODIFY

```

```

IMPORTING it_update FOR UPDATE Item.

METHODS delete FOR MODIFY
IMPORTING it_delete FOR DELETE Item.
ENDCLASS.

CLASS lhc_item IMPLEMENTATION.

METHOD read_item.
LOOP AT it_read INTO DATA(ls_key).
    SELECT SINGLE order_id, item_id, material_desc, quantity
    FROM zrap08_order_i
    WHERE order_id = @ls_key-order_id AND item_id = @ls_key-item_id
    INTO @DATA(ls_data).
    IF sy-subrc = 0.
        APPEND VALUE #(
            %key          = ls_key-%key
            order_id      = ls_data-order_id
            item_id       = ls_data-item_id
            material_desc = ls_data-material_desc
            quantity      = ls_data-quantity ) TO et_result.
    ENDIF.
ENDLOOP.
ENDMETHOD.

METHOD update.
" 留空——這一課的練習題，交給你自已補完（見下方「練習」段落）
ENDMETHOD.

METHOD delete.
" 留空——這一課的練習題，交給你自已補完（見下方「練習」段落）
ENDMETHOD.

ENDCLASS.

```

- **read_item** 一定要補上，不能留空：跟 **lhc_header** 的 **read_header** 是同一個道理——Unmanaged BDO 沒有框架自動生成的讀取邏輯，任何實體只要不補 **FOR READ Handler**，這個實體就完全查不到資料（Fiori Elements Preview 的 Item 表格會是空的、EML **READ ENTITIES ... ENTITY Item** 也拿不到東西），不像 **update / delete** 留空只是「這個操作不能用」、不影響其他功能。
- **update / delete** 留空是刻意的：**Ctrl+1** 生成的骨架本來就會依 BDEF 宣告的操作（Item 的 **update; / delete;**）長出這兩個空方法，這一步先留空即可，啟用時只會出現「operation not implemented」的警告，不影響這一課「驗證 Create-by-Association」的目標；把它們補完是下面「練習」段落要你自己做的事。

存檔 + 啟用整個類別（**lhc_header / lhc_item** 兩個類別都在同一份 Include 裡，一次存檔啟用即可，不用分開處理）。

8. 通知 Claude 驗證

完成以上步驟後跟 Claude 說一聲，Claude 會：

1. 讀取比對你建立的物件內容 (`sap_get_source` / GET 讀回)
2. 確認 `sap_inactive_objects` 沒有殘留
3. 建立一支驗證程式 (`ZR_RAP08_DEMO`，這是 `PROG/P`，不受這一課「物件由你建立」的限制，Claude 可以直接建)，用 `programrun` 跑一次 Create-by-Association，確認 Header + Item 真的一次建立成功

如果啟用過程中遇到跟這篇講義描述不一樣的錯誤訊息，把畫面截圖或錯誤文字貼給 Claude，會一起排查 (這系統過去也發生過「講義寫的步驟」跟「Eclipse 實際畫面」有落差的情況，回報後 Claude 會回頭修正講義)。

練習：延伸這個 Header-Item 結構

輪到你了：完成上面 8 個步驟建好基本結構，Claude 也驗證過 Create-by-Association 成功之後，試著自己補完 `lhc_header` / `lhc_item` 裡還留空的 `update` / `delete` 四個 Handler Method (`FOR MODIFY IMPORTING it_update FOR UPDATE Header / FOR UPDATE Item / it_delete FOR DELETE Header / FOR DELETE Item`)，讓這個 Header-Item BO 具備完整的 CRUD 能力。遇到不確定的語法，可以參考這一課 `create` / `create_item` / `read_header` / `read_item` 的寫法類推 (`UPDATE` / `DELETE` 都要記得 `WHERE` 條件帶對 Key 欄位——Item 是兩個 Key 欄位 `order_id` + `item_id`)，或直接問 Claude。

這一題有個容易漏掉的地方：使用者在 Fiori Elements Preview 實際點 Delete 測試時，因為 `lhc_header` 一開始完全沒有 `update` / `delete` 方法 (連宣告都沒有)，BDEF 雖然宣告了 `delete`；但框架找不到對應的 Handler 可以呼叫，Delete 按下去沒有任何效果、那一列還留在畫面上——這正好印證了這個練習的必要性：**BDEF 宣告的操作跟實際有沒有 Handler 實作，是兩件獨立的事**，宣告了但沒實作，這個操作在畫面上會「看起來存在但按了沒反應」，不會直接報錯。

解答

Claude 已經直接把下面的解答補進 `ZBP_I_RAP08_ORDER` (依使用者要求代為實作，不是留給讀者自己核對)，已啟用成功。

`lhc_header` 新增兩個方法宣告 + 實作：

```
METHODS update FOR MODIFY
  IMPORTING it_update FOR UPDATE Header.

METHODS delete FOR MODIFY
  IMPORTING it_delete FOR DELETE Header.
```

```
METHOD update.
  LOOP AT it_update INTO DATA(ls_update).
    UPDATE zrap08_order SET description = @ls_update-description
      WHERE order_id = @ls_update-%key-order_id.
  ENDLLOOP.
ENDMETHOD.
```

```

METHOD delete.
  LOOP AT it_delete INTO DATA(ls_delete).
    " Composition 的存在依賴關係：刪 Header 前要先刪掉所有底下的 Item，Unmanaged 不會自動
    連帶處理
    DELETE FROM zrap08_order_i WHERE order_id = @ls_delete-%key-order_id.
    DELETE FROM zrap08_order WHERE order_id = @ls_delete-%key-order_id.
  ENDLOOP.
ENDMETHOD.

```

⚠ **delete** 方法回答了思考題 3 (**Cascading Delete**)：查證官方文件的結論是——Unmanaged BDEF 完全沒有框架自動處理，`association _Item { create; }` 只宣告了「允許透過關聯建立子實體」這一件事，跟刪除完全無關；如果只刪 `zrap08_order`、不管 `zrap08_order_i`，會留下一堆 `order_id` 對應不到任何 Header 的孤兒 Item 資料列，破壞 Composition「子實體不能脫離父實體存在」的語意。所以 **delete** 方法要自己先刪子表、再刪父表（順序不能反過來，否則如果中途失敗，可能留下沒有 Header 但還有 Item 的髒資料）。

lhc_item 原本留空的 **update** / **delete** 補上實作：

```

METHOD update.
  LOOP AT it_update INTO DATA(ls_update).
    UPDATE zrap08_order_i
      SET material_desc = @ls_update-material_desc,
          quantity      = @ls_update-quantity
      WHERE order_id = @ls_update-%key-order_id
          AND item_id = @ls_update-%key-item_id.
  ENDLOOP.
ENDMETHOD.

METHOD delete.
  LOOP AT it_delete INTO DATA(ls_delete).
    DELETE FROM zrap08_order_i
      WHERE order_id = @ls_delete-%key-order_id
          AND item_id = @ls_delete-%key-item_id.
  ENDLOOP.
ENDMETHOD.

```

啟用後有三則警告 (`type="W"`，不影響啟用)：The operation "SAVER" for entity "ZI_RAP08_ORDER" is not implemented、The operation "READ" for association "_ITEM"/"_HEADER" is not implemented——第一個是沒有實作 `check_before_save` / `finalize` / `save` / `cleanup` 這組 Saver Hook（這一課用不到，屬於進階主題）；後兩個是沒有實作「透過關聯導覽讀取」（`READ ... BY \_Item / BY \_Header`），跟這裡用到的「直接對 Item / Header 各自單獨 READ」是不同的操作，這一課的驗證方式不需要它，但如果 Fiori Elements Object Page 的 Item 子表格顯示不出資料，這會是需要補的下一塊，不在這一課的必要範圍內，留給有興趣深入的人自己查證。

Part D：加碼——Service Definition / Service Binding，測試 Service / OData 層

前面「為什麼驗證用的是 ABAP Report」那一段提過，Behavior 層驗證成功之後，這一段才是真正要測 **Service / OData 層**——把這個 Header-Item BO 曝露成 OData Service，讓你可以用瀏覽器實際看到、操作它。

Service Definition 的語法：

- **expose <CDS View> as <別名>;**：把一個 CDS View 曝露成 OData 的 Entity Set，<別名> 是 OData 端看到的名稱。
- **⚠️ ⚠️ 已更正 (rap09 才發現、回頭修正這裡)**：**Composition** 子實體「不用另外 **expose**」這句話是錯的——原本這裡寫「只曝露 Root，Item 會透過導覽屬性自動可存取」，這是沒有實測驗證過的推論，被 rap09 的實測推翻。真相是：只 **expose Root** 的話，**\$metadata** 裡完全不會出現 **_Item** 的 **Navigation Property**，也沒有 **Item** 的 **EntityType** 定義——Fiori Elements Object Page 的 Item 子表格 Facet 會因此完全不渲染（連空表格都不會出現）。必須把子實體也一起 **expose**（可以取任何別名，不影響它「透過 Composition 被 Header 擁有」的語意），OData 端才會正確產生 **NavigationProperty / Association / AssociationSet**，Fiori Elements 才認得到這個子表格。完整的踩坑過程跟診斷方法（查 **\$metadata** 而不是猜）記錄在 rap09 講義。
- **⚠️ 命名要避開保留字**：Claude 一開始想用 **expose ZI_RAP08_ORDER as Order;**，啟用時報 **'Order' is a reserved keyword**——**ORDER** 在 CDS / SQL 裡跟 **ORDER BY** 衝突，改成 **Orders**（複數）就過了。這是個小提醒：Entity Set 別名撞到 SQL 關鍵字（**ORDER / GROUP / SELECT.....**）是常見的踩坑點。

Service Definition ZRAP08_SD（**Claude** 已建立並啟用，不需要你動手；下面是 rap09 更正後的最終版本，兩個實體都要 **expose**）：

```
@EndUserText.label: 'RAP08 Order Service Definition'
define service ZRAP08_SD {
  expose ZI_RAP08_ORDER as Orders;
  expose ZI_RAP08_ORDER_I as Items;
}
```

Eclipse ADT：建立並發布 Service Binding——換你動手

跟 rap04 教過的流程完全一樣，**Service Binding** 一律要你在 **Eclipse** 手動建立 + **Publish**——這是技術上的硬性限制（ADT REST API 手動建的 Service Binding 缺少精靈才會觸發的後端註冊步驟，Publish 永遠會失敗，見 [.claude/rules/sap-adt-mcp.md](#) 第 40.9 節），不是分工原則的選擇，Claude 這步真的做不到。

1. 對著 **ZRAP08_SD**（Service Definition）按右鍵 → **New Service Binding**（這系統的 ADT Plugin 已經把這個選項做成直接捷徑，不用繞 **Other ABAP Repository Object**）。
2. 填 **Name**（**ZRAP08_SB**）、**Description**、**Binding Type** 選 **OData V2 - UI**（不是 Web API——兩者差異、為什麼這門課一律選 UI，rap04「Service Binding：讓服務真正『活起來』」那一段已經詳細說明過，這裡不重複，忘記的話回去看那邊）→ **Next**。
3. **Select Transport Request** 畫面：**\$TMP** 套件會顯示「No change recording enabled for package \$TMP」，直接 **Finish** 即可，不用選傳輸請求。
4. 編輯器開啟後，點 **Publish** 按鈕——成功的話 **Local Service Endpoint** 狀態會變成 **Published**，**Unpublish** 按鈕會出現。
5. 點編輯器裡的 **Preview...** 按鈕，會開啟瀏覽器顯示 Fiori Elements List Report，應該會看到 **Orders** 這個清單（目前資料庫裡已經有 **ZR_RAP08_DEMO** 跑出來的 **ORD001** 測試資料，應該能看到這一筆）。

核心目標是「看到 **Publish** 成功、**Preview** 開得起來、資料讀得到」，證實 **Service / OData** 這一層真的通了——這才是跟前面 Behavior 層驗證互補的地方：Behavior 層證實了「BO 邏輯對」，這裡證實「曝露成 OData 之後，外部真的存取得到」。

加碼：Metadata Extension，讓畫面顯示得完整一點

第一次 Preview 出來大概是「陽春版」畫面：沒有 `@UI.*` Annotation 的話，List Report 的欄位標題會是 `ORDER_ID / DESCRIPTION` 這種大寫技術名稱（甚至完全看不到欄位，只有一個展開用的 > 箭頭），Object Page 點進去也看不到 Item 子表格——這是 rap02 已經教過的 `@UI.headerInfo` / `@UI.lineItem` / `@UI.selectionField` / `@UI.identification` / `@UI.facet` 這一整組標記在起作用，rap08 這兩個 CDS View 一開始沒有配。Claude 已經比照 rap02 的做法幫這兩個 CDS View 各建一份 Metadata Extension：

ZI_RAP08_ORDER 的 **Metadata Extension**（Header——除了一般欄位標記，還多了 `@UI.facet` 讓 Object Page 秀出 Item 子表格）：

```
@Metadata.layer: #CUSTOMER
@UI: {
  headerInfo: {
    typeName: 'Order',
    typeNamePlural: 'Orders',
    title: { type: #STANDARD, value: 'order_id' }
  }
}

annotate view ZI_RAP08_ORDER with
{
  @UI.facet: [
    { id: 'GeneralInformation', purpose: #STANDARD, type: #IDENTIFICATION_REFERENCE,
    label: 'General Information', position: 10 },
    { id: 'Items', purpose: #STANDARD, type: #LINEITEM_REFERENCE, label: 'Order
Items', targetElement: '_Item', position: 20 }
  ]

  @UI.lineItem: [{ position: 10 }]
  @UI.selectionField: [{ position: 10 }]
  @UI.identification: [{ position: 10 }]
  order_id;

  @UI.lineItem: [{ position: 20 }]
  @UI.identification: [{ position: 20 }]
  description;

  @UI.lineItem: [{ position: 30 }]
  @UI.identification: [{ position: 30 }]
  created_at;
```

```
@UI.lineItem: [{ position: 40 }]
@UI.identification: [{ position: 40 }]
created_by;
}
```

新語法：**@UI.facet** 的 **type: #LINEITEM_REFERENCE + targetElement** (查證 SAP 官方文件 **Using Facets to change the Object Page Layout** 確認) ——這是讓 Object Page 顯示「子實體表格」的標準做法：

- **type: #IDENTIFICATION_REFERENCE**：這個 Facet 顯示「標了 **@UI.identification** 的欄位」，預設抓的是同一個實體自己的欄位 (不用另外指定 **targetElement**) ——這裡用來顯示 Header 自己的基本資料。
- **type: #LINEITEM_REFERENCE + targetElement: '_Item'**：這個 Facet 顯示「**_Item** 這個關聯指向的實體裡，標了 **@UI.lineItem** 的欄位」，用表格格式呈現——這就是讓 Object Page 出現 Item 子表格的關鍵。**@UI.lineItem** 要標在 **ZI_RAP08_ORDER_I (Item)** 自己的 **Metadata Extension** 裡，不是標在 **Header** 這邊——Facet 只是「指去哪裡拿資料」，資料本身的欄位標記要在目標實體上定義。

ZI_RAP08_ORDER_I 的 **Metadata Extension** (Item——只需要 **@UI.lineItem**，因為它是被 Header 的 Facet 拉過去顯示的，不需要自己的 **headerInfo / facet**)：

```
@Metadata.layer: #CUSTOMER

annotate view ZI_RAP08_ORDER_I with
{
  @UI.lineItem: [{ position: 10 }]
  item_id;

  @UI.lineItem: [{ position: 20 }]
  material_desc;

  @UI.lineItem: [{ position: 30 }]
  quantity;
}
```

⚠ 踩到的坑：**ZI_RAP08_ORDER_I** 原本的 **CDS View** 缺了 **@Metadata.allowExtensions: true**——啟用 Metadata Extension 時報 Annotation 'Metadata.allowExtensions' missing in 'ZI_RAP08_ORDER_I'。**ZI_RAP08_ORDER** (Header) 從一開始的 DDL 就有這一行，但 **ZI_RAP08_ORDER_I** (Item) 當初漏了——這是 rap08 原始講義的第三個小疏漏 (前面已經修過 DDIC 型別漏用 Data Element、實作類別漏了 **lhc_item** 兩個)，已經回頭補進上面第 4 步的 DDL。這代表任何 **CDS View**，只要之後想幫它掛 **Metadata Extension**，都要記得先確認它本身有沒有 **@Metadata.allowExtensions: true**——這不是 Metadata Extension 檔案裡的內容，是被擴充的那個 **CDS View** 自己要開的開關，邏輯類似「你要允許被擴充，才能真的被擴充」。如果你的 **ZI_RAP08_ORDER_I** 是照舊版講義建的 (沒有這一行)，要補上這行、存檔、重新啟用，Claude 才能繼續啟用 Metadata Extension。

加碼：用 Postman 直接呼叫 OData，不透過瀏覽器/Fiori Elements

Preview 是透過瀏覽器打 OData、再由 Fiori Elements 框架渲染畫面；Postman 則是直接、更貼近底層地打同一組端點，兩者測的是同一個 Service / OData 層，只是後者能讓你清楚看到每個 HTTP Request/Response 長什

麼樣子。

前置：Service URL 固定是 `/sap/opu/odata/sap/<Service Binding 名稱>`，搭配這系統對外主機名稱 + Port (`erpdemo01.itts.com.tw:44300`，rap04 已教過)，這一課的 Base URL 是 `https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP08_SB` (下面每一步都以這個 Base URL 為前綴，完整網址直接照抄即可，不用自己拼接)。

1. **Authorization**：每個 Request 的 Authorization 分頁選 **Basic Auth**，填你平常登入這套系統的帳密——OData V2 走 HTTP Basic Auth，跟瀏覽器 Preview 跳出的登入視窗是同一套機制。
2. **GET 讀清單**：

``http GET `https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP08_SB/Orders?$format=json&sap-client=130`` \$format=json 是關鍵 (OData V2 預設回 XML Atom Feed)，sap-client 這系統要帶。回應的 `d.results`` 陣列就是訂單清單。

1. **GET \$metadata** (建議先做)：

``http GET `https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP08_SB/$metadata`` 列出 Orders EntitySet 的所有欄位跟導覽屬性 (Composition 出去的 `_Item`` 在 OData 端叫什麼名字)——要測 Item 或 Deep Insert (一次連 Header+Item 一起建) 之前，先查這份文件確認正確的導覽屬性名稱，不要用猜的。

1. **GET 單筆**：

``http GET `https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP08_SB/Orders('ORD001')?$format=json&sap-client=130`` OData V2 單筆存取語法是 EntitySet('KeyValue') (單一 Key) 或 EntitySet(key1='xxx',key2='yyy') (複合 Key)。

1. 取得 **CSRF Token** (寫入操作的必要前置)：新建一個 GET Request，網址可以沿用第 2 步的 `https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP08_SB/Orders?sap-client=130`，Header 加 `X-CSRF-Token: Fetch`，Send 後從回應的 **Headers** 找到 `x-csrf-token` 值複製下來——Postman 會自動把這次回應的 Session Cookie 存進它自己的 Cookie Jar，不用手動複製 Cookie (這條路走的是正常瀏覽器式 Session，不會遇到第 45 節提過的「Claude 自我呼叫」那個 App Server 親和性問題)。
2. **POST 新增**：

``http POST `https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP08_SB/Orders?sap-client=130`` Header 帶 `X-CSRF-Token + Content-Type: application/json`，Body：`{ "order_id": "PM001", "description": "Postman Test Order" }`——成功回 **201 Created**，會呼叫到 `lhc_header.create``。

1. **PUT 更新**：

``http PUT `https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP08_SB/Orders('PM001')?$format=json&sap-client=130`` Body：`{ "description": "Postman Test Order - Updated" }`——成功回 **204 No Content**，呼叫到剛補上的 `lhc_header.update``。

1. **DELETE 刪除**：

``http DELETE

https://erpdemo01.itts.com.tw:44300/sap/opu/odata/sap/ZRAP08_SB/Orders('PM001')?sap-client=130 ` 只需要 X-CSRF-Token——成功回 204 No Content · 呼叫到 lhc\_header.delete` (含 Cascading Delete · 連帶清掉這筆訂單底下的 Item) 。

小提醒：CSRF Token 是同一個 Session 內共用的，不用每次寫入都重新 Fetch，除非過期（回 403）；建議用 Postman 的 Collection + 環境變數存 base URL / Token，不用每個 Request 重打網址。

Part C：這一課的關鍵發現總結

發現	內容
Composition / Association 宣告語法	Managed / Unmanaged 完全一樣，都支援這系統的舊式 define view (不需要 view entity)
子實體 lock 子句	這系統要 lock dependent (欄位 = 欄位)，不是官方教材的 lock dependent by _別名
Create-by-Association Handler 簽章	FOR CREATE <alias>_<association> ，反斜線跳脫底線開頭的關聯別名
子實體 readonly 欄位	使用者需要輸入的欄位（如子實體自己的 Key）不能標 readonly ，只有框架/邏輯決定值的欄位才能標
EML 呼叫端父子關聯寫法	父實體 Key 是使用者自訂值時，用 %key-<欄位> ；只有父實體 Key 是框架自動編號時才需要 %cid_ref ——這系統上這個選錯完全不會報錯，只是靜默地建不出子實體，是這一課最容易踩的坑
這系統能不能真正執行	Managed：❌ (Managed Runtime 白名單限制)；Unmanaged：✅ 已驗證成功

學習目標

- 能寫出 Composition / Association 的 CDS 語法：**composition [0..*] of ... as _<別名>** (父)、**association to parent ... as _<別名> on ...** (子)
- 能寫出這系統適用的多實體 BDEF：一個 BDEF 檔案含多個 **define behavior for** 區塊，子實體用 **lock dependent** (欄位 = 欄位)
- 能寫出 Create-by-Association 的 Handler Method：**FOR MODIFY IMPORTING it_create FOR CREATE <alias>_<association>**，處理雙層 LOOP (外層父實體、內層 **%target** 子實體)
- 知道 EML 呼叫端父實體 Key 是使用者自訂值時要用 **%key-<欄位>** 連結子實體建立，不是官方範例常見的 **%cid_ref** (那是給框架自動編號情境用的)
- 能在 Eclipse ADT 完整建立一組 Header-Item 兩層結構的 RAP 物件：兩張表、兩個 CDS View (Composition + Association to Parent)、一個 BDEF (含兩個 **behavior** 區塊)、一個實作類別
- 能講出子實體欄位的 **readonly** 判斷原則：使用者要輸入的欄位不能 **readonly**，框架/邏輯自動決定的才能
- 知道 **RAP** 測試分兩個獨立的層次：Behavior / BO 層 (用 EML，不需要 Service Definition / Binding) 跟 Service / OData 層 (真正的 OData 呼叫，需要 Publish)，兩層要分開驗證、分開排錯

- 能寫出 Service Definition 曝露 Composition Root 的語法：`expose <CDS View> as <別名>;`，知道子實體不用另外 `expose`（透過導覽自動可存取），也知道別名要避開 SQL 保留字（如 `ORDER`）

物件清單

物件	名稱	型別	建立方式	可執行性
Domain / Data Element	ZRAP08_ORDERID	DOMA/DD + DTEL/DE	Claude 建立並啟用（課程獨有業務概念，比照 rap02 task_id 模式）	—
Domain / Data Element	ZRAP08_ITEMID	DOMA/DD + DTEL/DE	Claude 建立並啟用	—
Domain / Data Element	ZRAP08_QUANTITY	DOMA/DD + DTEL/DE	Claude 建立並啟用	—
Header 表格	ZRAP08_ORDER	TABL/DT	使用者 Eclipse 建立	✓ 已驗證
Item 表格	ZRAP08_ORDER_I	TABL/DT	使用者 Eclipse 建立	✓ 已驗證
Header CDS View	ZI_RAP08_ORDER	DDLS/DF	使用者 Eclipse 建立	✓ 已驗證
Item CDS View	ZI_RAP08_ORDER_I	DDLS/DF	使用者 Eclipse 建立	✓ 已驗證
Behavior Definition (Unmanaged)	ZI_RAP08_ORDER	BDEF/BDO	使用者 Eclipse 建立	✓ 已驗證
實作類別	ZBP_I_RAP08_ORDER	CLAS/OC	使用者 Eclipse 建立（Ctrl+1 生成骨架，含 lhc_header / lhc_item 兩個 Handler 類別）	✓ 已驗證
EML 驗證程式	ZR_RAP08_DEMO	PROG/P	Claude 建立（PROG 不受本課分工限制）	✓ 已用 programrun 驗證成功
Service Definition	ZRAP08_SD	SRVD/SRV	Claude 建立並啟用（ <code>expose ZI_RAP08_ORDER as Orders;</code> ）	✓ 已啟用
Service Binding	ZRAP08_SB	SRVB/SVB	使用者 Eclipse 建立 + Publish（技術上必須，見 Part D）	待使用者建立後 Claude 驗證
Metadata Extension (Header)	ZI_RAP08_ORDER	DDLX/EX	Claude 建立並啟用（依使用者要求加碼，含 @UI.facet 顯示 Item 子表格）	✓ 已啟用

物件	名稱	型別	建立方式	可執行性
Metadata Extension (Item)	ZI_RAP08_ORDER_I	DDLX/EX	Claude 建立並啟用	✅ 已啟用

Claude 用來驗證語法的暫時性物件（不算課程正式物件，語法已確認正確，過程記錄在 `.claude/rules/sap-adt-mcp.md`）：`ZRAP08V_H / ZRAP08V_I`（表格）、`ZI_RAP08V_H / ZI_RAP08V_I`（Managed CDS + BDEF）、`ZI_RAP08VU_H / ZI_RAP08VU_I`（Unmanaged CDS + BDEF）、`ZBP_I_RAP08VU_H`（Unmanaged 實作類別）、`ZR_RAP08V_DEMO`（驗證程式，已確認 Create-by-Association 端對端成功）。

驗證方式

Behavior / BO 層：

1. 你依照上方 Eclipse Step by Step 建立 `ZRAP08_ORDER / ZRAP08_ORDER_I / ZI_RAP08_ORDER / ZI_RAP08_ORDER_I / ZI_RAP08_ORDER`（BDEF）/`ZBP_I_RAP08_ORDER`
2. 通知 Claude，Claude 讀取比對內容、確認 `sap_inactive_objects` 無殘留
3. Claude 建立 `ZR_RAP08_DEMO` 並用 `programrun` 執行，驗證 Create-by-Association 一次呼叫同時建立 Header + Item 成功

✅ 已驗收（2026-08-17）：使用者依照上方步驟在 Eclipse 建立全部六個物件，Claude 讀取比對六個物件內容，逐一跟講義程式碼完全相符（含 `lhc_item` 的 `read_item / update / delete` 三個方法，`update / delete` 依講義設計保持留空），`sap_inactive_objects` 確認 0 筆殘留。Claude 建立 `ZR_RAP08_DEMO` 並用 `programrun` 執行，輸出：

```
after commit entities
header found: ORD001      Demo Order
item rows found: 2
0010 Widget A   5.00
0020 Widget B   3.00
```

Service / OData 層（Part D，加碼）：

1. Claude 已建立並啟用 Service Definition `ZRAP08_SD`（`expose ZI_RAP08_ORDER as Orders;`）
2. 你依照 Part D 的 Eclipse Step by Step 建立 Service Binding `ZRAP08_SB`（OData V2 - UI）並 Publish
3. 用 Eclipse 編輯器的 **Preview...** 開啟 Fiori Elements List Report，確認能看到 `Orders` 清單（含 `ZR_RAP08_DEMO` 建立的 `ORD001` 測試資料）
4. 通知 Claude（截圖或文字描述皆可），Claude 會讀取 `ZRAP08_SB` 的 `srvb:published` 狀態確認 Publish 成功

Header + 兩筆 Item 一次呼叫建立成功，且透過 `READ ENTITIES` 分別讀回 Header / Item 驗證資料正確——這也連帶驗證了 `lhc_item.read_item`（本課程講義修正後才補上的方法）確實有效，沒有它 `item rows found` 會是 0。

思考題

1. 這一課的 `create_item` 方法用兩層 LOOP (外層父實體、內層 `%target`)。如果 EML 一次呼叫要對**兩個不同的 Header** (例如 `ORD001` / `ORD002`) 分別建立各自的 Item，這段程式碼不用改就能處理嗎？為什麼？
2. `%key-order_id` (這一課用的，父 Key 是使用者自訂值) 跟 `%cid_ref` (官方範例常見，父 Key 是框架自動編號) ——如果 `rap02` 的 `ZI_RAP02_TASK` 改成用 `numbering:managed` 自動產生 `task_id`，Create-by-Association 建立子實體時該用哪一種？
3. Composition 的父子關係也隱含「Cascading Delete」(刪除父實體，子實體一起被刪除) ——這一課的 BDEF 只宣告了 `association_item { create; }`，沒有另外宣告 `delete` 相關的關聯操作。查證官方文件 `ABENBDL_ASSOC_STAND_OPS`，Cascading Delete 是不是需要額外宣告，還是父實體 `delete;` 就自動連帶處理？

答案

這一課的主要物件由使用者在 Eclipse 建立，Claude 事後驗證，已於 2026-08-17 驗收完成。答案物件快照已存入 `src/ABAP_Training_RAP/`：`zrap08_order.tabl.abap` / `zrap08_order_i.tabl.abap` / `zi_rap08_order.ddls.abap` / `zi_rap08_order_i.ddls.abap` (含後補的 `@Metadata.allowExtensions: true`) / `zi_rap08_order.bdef.abap` / `zbp_i_rap08_order.clas.abap` (Global 類別本體) / `zbp_i_rap08_order.clas.locals_imp.abap` (Local Types Include，含 `lhc_header` / `lhc_item` 兩個 Handler 類別) / `zr_rap08_demo.prog.abap` (EML 驗證程式)。另外三組 Domain / Data Element (`ZRAP08_ORDERID` / `ZRAP08_ITEMID` / `ZRAP08_QUANTITY`) 已存為 `.doma.xml` / `.dtel.xml`；Part D 加碼的 Service Definition (`zrap08_sd.srvd.abap`) 與兩份 Metadata Extension (`zi_rap08_order.ddlx.abap` / `zi_rap08_order_i.ddlx.abap`) 也都已存入。